# Design patterns and backend architecture

**Chapter 6: Fast comparisons interviewers ask**

Practice designs, not claims about an employer.

## 6. Fast comparisons interviewers ask

These vertical comparisons avoid landscape-only tables. Use the final question as your interview decision rule.

### Comparison 1: Strategy vs State

**First concept:** Strategy selects interchangeable rules, such as domestic and cross-border fees.

**Second concept:** State limits behavior based on current lifecycle status, such as CREATED or SETTLED.

**Decision question:** Am I choosing a rule, or validating a state transition?

### Comparison 2: Adapter vs Facade

**First concept:** Adapter translates a legacy provider protocol into the application contract.

**Second concept:** Facade presents one use-case entry point to several collaborators.

**Decision question:** Am I translating an incompatible interface, or simplifying orchestration?

### Comparison 3: Decorator vs Proxy

**First concept:** Decorator adds composable behavior, such as measuring call latency.

**Second concept:** Proxy controls access or lifecycle, such as Spring transaction advice.

**Decision question:** What purpose does the wrapper serve?

### Comparison 4: Factory vs Builder

**First concept:** Factory chooses or creates a concrete handler.

**Second concept:** Builder assembles one complex request with optional parts.

**Decision question:** Is selection the problem, or safe construction?

### Comparison 5: Layered vs Hexagonal

**First concept:** Layered assigns controller, business, and persistence responsibilities.

**Second concept:** Hexagonal requires the core to own ports, with technology at the edges.

**Decision question:** Do infrastructure dependencies leak into the core?

### Comparison 6: Modular monolith vs microservices

**First concept:** Modular monolith has one deployable with internal capability boundaries.

**Second concept:** Microservices have independent deployments and explicit network/data boundaries.

**Decision question:** Will independent operations outweigh distributed complexity?

## Comparison 7: Event vs command

**First concept:** Event states a completed fact: PaymentApproved.

**Second concept:** Command requests an action: ApprovePayment.

**Decision question:** Has the business change happened already?

## Comparison 8: Local transaction vs saga

**First concept:** Local transaction gives atomic changes within its database scope.

**Second concept:** Saga coordinates independently committed steps and compensations.

**Decision question:** Can one transaction own all affected state?

## Comparison 9: CQRS vs event sourcing

**First concept:** CQRS separates read and write models.

**Second concept:** Event sourcing keeps events as the primary durable state history.

**Decision question:** Is the need separate query shape, or event replay as source of truth?

## Comparison 10: Retry vs circuit breaker

**First concept:** Retry repeats a *safe* operation after a transient failure.

**Second concept:** Circuit breaker stops calls when continued failures are likely.

**Decision question:** Is an extra attempt safe and likely to help?

## Comparison 11: Bulkhead vs rate limit

**First concept:** Bulkhead isolates internal concurrency/resource pools.

**Second concept:** Rate limit restricts incoming request volume or quota.

**Decision question:** Which boundary needs protection?

## Comparison 12: Cache vs read model

**First concept:** Cache stores a faster copy of the same data.

**Second concept:** Read model reshapes data for a specific query/use case.

**Decision question:** Is the need faster access or a different view?